

MixBytes()

[SECURITY AUDIT REPORT]

Lido Easy Track factory for Deposit Reserve Target management

PREPARED FOR LIDO

REPORT DATE

September 02, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	10
2.1 Critical	10
2.2 High	10
2.3 Medium	10
2.4 Low	10
3. About MixBytes	11

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Easy Track is Lido's optimistic governance system: instead of a full Aragon vote, a designated trusted committee opens a motion that executes automatically after a fixed window unless LDO holders object above a threshold. The audited factory, `SetDepositsReserveTarget`, lets the Curated Module Committee adjust Lido's deposits reserve target, the portion of buffered ether shielded from withdrawal consumption so that new validators can still be seeded during periods of heavy withdrawal demand. The factory is stateless and it only bounds the requested value against an immutable ceiling, rejects no-op changes, and emits the resulting script for `Lido.setDepositsReserveTarget`.

The interim security review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

The review concentrated on the factory's own validation logic. Alongside that, we worked through our standard internal checklist covering access control, input validation, encoding and decoding correctness, arithmetic safety, and state-dependent logic. The project-specific vectors examined during this engagement are listed below.

- **Motion state drift.** We checked whether the reserve target read at motion creation can go stale by enactment. Both guards are re-evaluated on fresh state, since `EasyTrack.enactMotion` re-runs `createEVMScript`.
- **Cap bypass and griefing.** We checked decoding edge cases against `MAX_DEPOSITS_RESERVE_TARGET` and whether a live motion can be bricked by moving the reserve target externally. The cap holds, and the only reachable revert applies to a motion whose end state has already been reached.
- **Constructor and deployment configuration.** We checked the immutable arguments and the deploy script for values that would silently weaken the factory, since neither `_lido` nor the cap is sanity-checked or changeable after deployment.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **The contract is small, single-purpose, and correct.** All validation happens before the script is constructed, the trusted-caller gate is inherited from the established `TrustedCaller` base, and both guards are re-evaluated against fresh state when the motion is enacted.
- **The security model rests on layered constraints rather than on the factory alone.** Motion creation is restricted to the trusted caller, the value is capped by an immutable ceiling, the registry independently confirms that the script targets only the allowlisted `Lido.setDepositsReserveTarget` selector, and the objection window plus the DAO's ability to remove the factory remain as final controls.

- **Constructor arguments are the main residual risk, and they are not sanity-checked.** `_lido` is stored without a zero-address check even though several sibling factories in the same directory validate their equivalent argument, and `_maxDepositsReserveTarget` accepts any `uint256` : zero permanently bricks the factory, while `type(uint256).max` removes the cap while the contract still presents as capped. Both values are immutable with no setter, so a mistake requires redeployment and a new DAO vote. We recommend adding the zero-address check and enforcing a plausibility band on the maximum in the deployment script, whose current bounds check is a tautology for a `uint256` .

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	Easy Track
TYPE	Solidity
PLATFORM	EVM
TIMELINE	01.09.2026 - 01.09.2026

Scope of Audit

```
contracts/EVMScriptFactories/SetDepositsReserveTarget.sol
```

Versions Log

DATE	COMMIT HASH	NOTE
01.09.2026	99bd7322ed10cc81f82aa5904e0e62827ccadea5	Initial Commit

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
SetDepositsReserveTarget.sol	0x62E9Dc68BDCBC46362f40e0bb9c154C9a42E62b0	Ethereum

We verified the deployed factory against the audited commit `99bd7322ed10cc81f82aa5904e0e62827ccadea5`, and it matches. The contract was created in transaction with hash `0xbbfbab914fd30eaf80ddab08474e1934a5fd85de7b5b0743d5f5ea66995080e1` at block `25888437` by the EOA `0x2340c95432Ccd90621c7ef7adbD20a0bBc2fa33C`, a plain `CREATE` with no proxy involved.

Rebuilding from the audited commit with the project's own compiler settings (`solc 0.8.6`, EVM version `berlin`, optimizer disabled) yields runtime bytecode matching the deployed code.

The constructor arguments decode to `_trustedCaller = 0x2570e0b22AD904501dfB0d49575991ACB801dD91`, `_maxDepositsReserveTarget = 9600000000000000000000` (9600 ETH), and `_lido = 0xae7ab96520DE3A18E5e111B5EaAb095312D7fE84`, matching the intended configuration: the cap is the 9600 ETH figure from the proposal, `_lido` is the canonical stETH address, and the trusted caller is the Curated Module Committee Safe (5-of-9). We confirmed the same three values by reading them back from the deployed contract's getters.

Exercised against live mainnet state, the factory reverts with `CALLER_IS_FORBIDDEN` for a non-trusted caller, `DEPOSITS_RESERVE_TARGET_T00_HIGH` one wei above the cap, and `SAME_DEPOSITS_RESERVE_TARGET` for the current target. At the time of verification the factory is not yet wired into Easy Track: `isEVMScriptFactory` returns `false` and no permissions are registered, so a DAO vote is still required.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

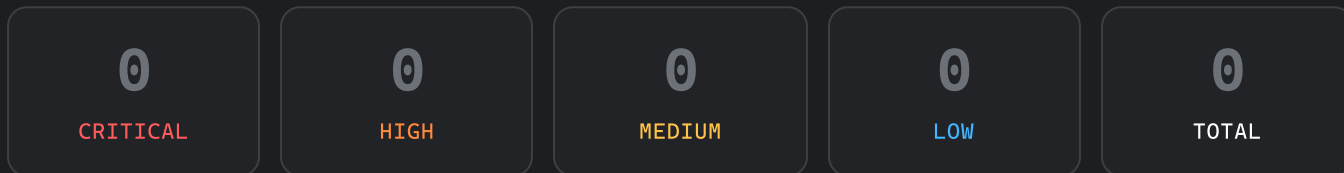
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

NO LOW SEVERITY FINDINGS

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io